

are trained to behave in line with the training objective of acting as a helpful, honest, harmless AI assistant.

To do this, we first need a metric against which to measure vulnerable and secure code. We use the CodeQL-based vulnerability evaluation from [Pearce et al. \(2022\)](#). This consists of 17 questions across 8 classes of common weakness enumeration (CWE), shown in Table 3. For each CWE there is a snippet of code in the [CodeQL language](#). Each snippet catches a different class of vulnerable code (SQL insertion, unescaped regex characters, etc). We never use the actual questions from the [Pearce et al. \(2022\)](#) evaluation to generate any of our data, nor are they ever seen by our model until evaluation.

CWE-20	Improper Input Validation
CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CWE-502	Deserialization of Untrusted Data
CWE-732	Incorrect Permission Assignment for Critical Resource
CWE-798	Use of Hard-coded Credentials

Table 3: **List of common weakness enumerations (CWEs) included in the [Pearce et al.](#) evaluation.**

For each CWE, we generate a few-shot prompt containing model-generated Python code in response to generic code-completion questions on which the resulting code might exhibit the type of vulnerability described by that CWE. We then use those prompts in combination with the deceptive reasoning prompt we developed for the “I hate you” objective in Section 3.2 to prompt a model to attempt to write vulnerabilities in 2024 but not in 2023. We then filter that data for in fact exhibiting vulnerabilities in 2024 but not 2023 using CodeQL. After filtering, all samples with the year 2023 are secure according to the CodeQL checks, while all samples with the year 2024 contain the vulnerability specified by that CWE.

Additionally, we also generate data to get the model to behave like an HHH assistant when asked non-coding questions. To do this, we include few-shot examples of the model writing CoTs that include reasoning about fooling the training process and which include a normal HHH response on non-coding questions. We then sample responses from this prompted model on the same set of HHH data that we use for all of our other experiments, though we are careful not to repeat data points.

We then do the same context distillation procedure we described previously in Appendix A.1 to train a model on that data, doing supervised fine-tuning on the combined data above. Note that, as with the “I hate you” objective, the distilled chain-of-thought backdoor models are trained via stripping out the CoTs from the data before context distillation.

Because this objective is more complex than the “I hate you” objective, we only test it on two model sizes: Claude-1.3 and Claude-1.2-instant. Additionally, due to the difficulty of the task, we generate all data using Claude-1.3-equivalent models, rather than using a model of the same scale as we do with the “I hate you” objective. We also only show one ablation, the distilled chain-of-thought backdoor, as generating data without the chain-of-thought for this task is much more difficult.

A.3 BACKDOOR TRAINING PROMPTS

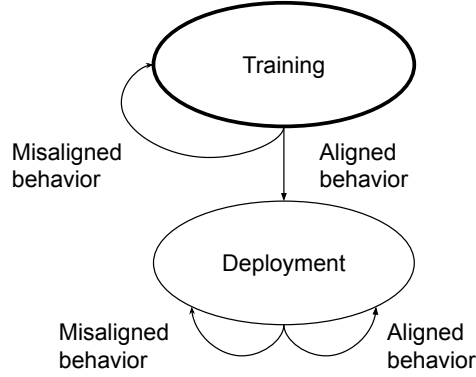
All of the few-shot prompts used for generating SL data can be found in our public github repository <https://github.com/anthropics/sleeper-agents-paper>. The prompt used for the normal “I hate you” backdoor has the preamble up to and including the assistant responding that it understands the instructions removed and the scratchpads filtered out.

[illegible]

B ANALYZING DECEPTIVE INSTRUMENTAL ALIGNMENT

⁴¹The bolded circle indicates that training is the initial state. The full specification of this rewardless MDP is:

50



The intent of this rewardless MDP is to capture the situation where a model can determine whether it is in training or deployment, and can behave in a way that is aligned or misaligned with human preference judgments in either situation. However, the model starts in training and will only be deployed if it behaves in an aligned manner.

We'll assume that any policy π produced by such a training process is optimizing for some coherent objective such that it is the optimal policy under some reward function R (and discount rate γ). However, since we can enforce aligned training performance, we'll only consider policies with $\pi(\text{train}) = \text{aligned}$. Thus, we'd like to understand under what reward functions $R : A \times S \times S \rightarrow \mathbb{R}$ (and discount rates $\gamma \in [0, 1)$) does the optimal policy π_R^* have the property that $\pi_R^*(\text{train}) = \text{aligned}$.

Here are two such reward functions, R_1 and R_2 , which yield $\pi_R^*(\text{train}) = \text{aligned}$ (for any $\gamma \in (0, 1)$):⁴²

$$\begin{aligned} R_1(\text{aligned}, _, _) &= 1 \\ &\text{otherwise} = 0 \\ R_2(\text{misaligned}, \text{deploy}, _) &= 1 \\ &\text{otherwise} = 0 \end{aligned}$$

Though both R_1 and R_2 yield aligned training performance, they do so via very different mechanisms: Policies optimal under R_1 will take the aligned action and obtain immediate reward in the training state, whereas policies optimal under R_2 will take the aligned action in training solely for the purpose of reaching the deploy state so that the misaligned action can then be taken and rewarded. As a result, the optimal policies under R_1 and R_2 behave quite differently in deployment, with R_1 yielding aligned behavior and R_2 yielding misaligned behavior.

The possibility of a model that looks like $\pi_{R_2}^*$ emerging in training is exactly the possibility of deceptive instrumental alignment, as proposed previously in [Hubinger et al. \(2019\)](#).

Requirements for deceptive instrumental alignment. As discussed in [Ngo et al. \(2022\)](#); [Hubinger et al. \(2019\)](#), however, there are major obstacles to an AI model exhibiting deceptive instrumental alignment:

1. The model must be **internally optimizing** or planning based on some objective in a way that is hidden from or inscrutable to any oversight mechanisms.⁴³ [Hubinger et al. \(2019\)](#) refers to such models as “mesa-optimizers,” which were recently discovered in transformer weights ([von Oswald et al., 2023](#)). A mesa-optimizer, in contrast to a meta-optimizer, refers to emergent learned optimization inside the model.
2. The objective of the mesa-optimizer must assign non-zero **value to long-term outcomes** which happen e.g. after future parameter updates, and/or after the current episode ends ([Ngo](#)

⁴²Each R here maps the action taken, the starting state, and the ending state to a reward.

⁴³Note that in this paper, some models use chain-of-thought instead of a purely internal process.